Nov 2 — 5 papers

Episode 45 · November 2, 2026 · 5 papers

Soei: Structured Observability of Evidence and Intent for Multi-Agent AI Software Engineering

SKIM

Mohammed Nayeem Khan, K S Sarun Kumar

An audit of what AI coding agents actually do across 45 real sessions, finding that roughly a quarter of code-editing episodes end with no check after the last edit, plus a proposed tamper-evident ledger that keeps an agent's claim of being done separate from proof that it is done.

PROBLEM When AI coding agents work for long stretches with little supervision, a developer coming back cannot easily tell what was done, which agent did it, or whether the task is really finished. Existing local tools like Agenttrail log events, but they do not separate a claim of completion from evidence, and they do not reliably link work that continues across sessions.

METHOD The authors first check the open-source Agenttrail tool against its source code line by line, and find two event pipelines that are never merged and a local mutation endpoint with no authentication. They then run a prototype over real agent transcripts, using a classifier whose labels were checked in a blind audit, to measure missing verification steps and session hand-offs. Their proposed system, Soei, is a hash-chained local event ledger with acknowledgement-aware events, a provenance graph aligned with the W3C PROV standard, and a layer that records a claim of completion separately from proof of it.

RESULTS Across 45 sessions with about 102 thousand records and 20 thousand tool calls, they estimate that 23.4 percent of code-editing episodes, with a confidence interval of 17 to 36 percent, end with no independent check, and that 57.8 percent of to-dos marked complete had no proof in between. The classifier's precision was 83 and 97 percent on its two labels, and the ledger is 20 percent of raw size and verifies about 126 thousand events per second.

CAVEAT The corpus is 45 sessions with no stated count of users, projects or agent products, so the headline rates may reflect one team's habits rather than agent behaviour in general. The abstract also gives no evidence that Soei helps a developer make better decisions, so the system is checked for throughput and tamper detection but not for usefulness to the people it is meant to serve.

WHY IT MATTERS This is off-topic for your work. It is about logging and auditing software agents, not vehicles, and it has nothing on negotiation, maneuver messages or trajectory coordination. The only loose link is the idea of keeping a claimed intent separate from verified completion in an acknowledgement-aware log, which echoes proposal and acceptance message design, but you will find no method here to carry over.

arXiv:2610.39468v1 PDF incremental

Kairos: Semantics-Aware Orchestration of Planning Engines across Classical, Numeric, Temporal and Hybrid PDDL

SKIM

Francisco Martín Rico et al.

Kairos is a portfolio planner that reads which PDDL features a task uses, picks among five specialised engines, races them under one shared time and memory budget, and only returns plans an independent validator has accepted.

PROBLEM Robots that re-plan at run time need one planner that accepts the full range of PDDL people actually write, from plain STRIPS through durative actions, numeric resources and hybrid PDDL+, and returns fast, trustworthy answers. Today each specialised planner covers only part of that range, and they disagree on what a failure or an unsolvable verdict actually means.

METHOD A profiler inspects each task's semantics to decide which of five engines apply: ground heuristic search, compressed temporal search, snap-action temporal search, time-step discretisation for hybrid dynamics, and lifted search. It then schedules them under a single budget. Every candidate plan goes through a validator that does not depend on the planner's own compilations. Unsolvable is reported only when a complete search has exhausted its space, so a timeout is never passed off as a proof.

RESULTS Across 1,756 tasks from nine comparative suites, Kairos solves more tasks at the full limits than the strongest planner in each suite, and it also solves the most tasks at 100 milliseconds and at one second. Domain-level resampling puts the gain above zero in six of the nine suites, and on classical planning against LAMA the difference is within noise. On 26 one-feature probes it handles all seventeen PDDL features tested, while no other planner handles more than eight.

CAVEAT This is portfolio engineering, not a new search algorithm, so the wins come from coverage and scheduling. In three of nine suites the gain is not distinguishable from zero. The 450 feature tasks and 26 probes were written by the authors, so the seventeen-versus-eight feature claim partly measures their own choice of features. The run-time robotics motivation is argued but never tested, since all evaluation is offline benchmark solving with no closed-loop re-planning, no dynamic replanning latency under execution, and nothing multi-agent.

WHY IT MATTERS This sits on the edge of your work. It is single-agent symbolic task planning, with no communication, negotiation, multi-vehicle conflict resolution or trajectory optimisation. The only useful point for you is the validated-answer discipline: only certified plans, and unsolvable claimed only on proof. That maps loosely onto how a maneuver protocol should separate a confirmed agreement from a timeout. If you ever use PDDL+ to model merge or platoon-join logic with continuous dynamics, it is worth knowing this exists.

arXiv:2610.39454v1 PDF incremental

Ask to Evolve: A Unified Harness for Lifelong Embodied Navigation with Hybrid Action Space and Persistent Memory

SKIM

Kai Sheng et al.

An LLM-driven indoor navigation agent that can ask a human for help and remembers places between visits, reporting near-perfect success on small subsets of standard embodied navigation benchmarks.

PROBLEM Embodied navigation agents usually act alone and start each episode with no memory, so they cannot resolve ambiguous instructions by asking or reuse what they learned on earlier visits. The authors argue that action, interaction and memory have been studied separately and never combined in one agent.

METHOD They wrap a multimodal model in a harness with four kinds of action: waypoint navigation, small discrete moves, choosing a target pixel in the image, and navigating to a named object. The agent can stop and ask a question when it is confused or stuck. Observations and dialogue are stored in a spatial memory tied to locations and retrieved on later visits.

RESULTS On four 100-episode sets from vision-language navigation and object navigation, first-visit success is 100 percent, with an average of 1.87 questions per episode. Full-sequence success is 98 percent on 50 LHPR-VLN sequences and 94 percent on 50 GOAT-Bench sequences, and success weighted by path length on R2R revisits rises from 82.23 to 93.31 percent with memory; the abstract names no baselines and reports a multi-day physical robot demo only qualitatively.

CAVEAT Perfect success on 100-episode subsets is a red flag. The questions are presumably answered by an oracle or a human with privileged knowledge, so the gain may come from leaked ground truth rather than better navigation. A reviewer would want the full benchmark splits, success with questions turned off, a fair comparison against baselines given the same help budget, and a statement of who answers the questions and what they know.

WHY IT MATTERS This is peripheral to your work. It covers one indoor robot asking a human for help, not several vehicles negotiating maneuvers, and there is nothing on message design, packet loss, platoons or right of way. The only loose connection is the idea of asking only when ambiguous, which loosely mirrors event-triggered intent requests, but the paper offers no protocol or analysis you could transfer.

arXiv:2610.39446v1 PDF incremental

Utility-WAM: Correcting Future-Induced Action Changes

SKIM

Feihong Zhang et al.

When a robot policy that imagines video of the future acts worse than the same policy acting without it, this paper trains a small residual adapter to undo the damage and a gate that skips video generation when it would not help.

PROBLEM World action models condition robot actions on generated visual futures, but averaged benchmark scores hide that this foresight barely helps overall and badly hurts specific precision tasks, all while nearly doubling inference latency. Nobody had isolated the action difference caused by the imagined future as the thing to measure and fix.

METHOD They compare the future-conditioned and future-free execution paths of one frozen model and treat the gap between their actions as the object to correct. A residual adapter is trained to pull that gap toward ground-truth dynamics, with a penalty whenever its error exceeds the future-free baseline. A lightweight gate, placed before generation, uses counterfactual action outcomes to decide when to skip video generation entirely.

RESULTS Imagined futures shift average success by only plus 0.8 points on LIBERO and minus 0.8 on RoboTwin 2.0 while costing 1.7 to 1.9 times the latency, but they cost Press Stapler 54 points and Place Can Basket 34; the adapter cuts those losses to 6 and 14. The gate skips generation on 75.5 percent of RoboTwin queries, cutting latency by 38 percent, and the system reaches 93.3 percent on RoboTwin, 98.9 percent on LIBERO, and 53 of 60 real bimanual trials against 43 for FAST-WAM.

CAVEAT The headline failure is told through two cherry-picked tasks, and the adapter still leaves a 14 point loss on one of them. The gate learns from counterfactual outcomes that may require running both paths during training, so its cost and its transfer to new tasks are unclear. The real-robot evidence is three tasks at 20 trials each, where a 10 trial gap against a single baseline is suggestive but statistically thin.

WHY IT MATTERS This is robot manipulation, not connected vehicles, and none of it involves multiple agents, communication, negotiation or right of way, so it sits outside your work. The only transferable idea is methodological: measure what extra predicted context actually changes in the chosen action, and gate it when it does not pay. That is a fair lens for asking when shared intent or trajectory messages are worth their bandwidth and latency, but you can get that idea from this abstract alone.

arXiv:2610.39367v1 PDF incremental

ProLoop: Progress-Aware Closed-Loop Framework via Structured State Relations for Long-Horizon Robotic Task Planning SKIM

Runxin Zou et al.

ProLoop has an LLM household-robot planner check each subgoal against a learned monitor that compares the desired state with the actual state, instead of guessing progress from the action log, and it gains about 13 points on ALFWorld.

PROBLEM LLM planners for long-horizon tasks usually infer whether a subgoal is done from the raw interaction history, so a wrong progress estimate quietly sends execution down the wrong path and early errors compound. Explicit progress tracking has been missing because LLM agents rarely keep a structured representation of the state they want to reach.

METHOD Each subgoal is written as a set of desired state relations, such as an object being inside a receptacle. A learned State-Relation Monitor checks each relation against the observed execution state. Those checks are combined into one of three labels: ongoing, deviated, or completed. The planner then uses that label to keep going, re-plan, or move to the next subgoal.

RESULTS On AgentBoard ALFWorld and a real robot platform, the authors report a 12.94 percent gain in success rate overall and 15.46 percent on Hard tasks, with 32.3 percent fewer environment interactions. The abstract does not name the baselines, does not say whether the gains are absolute or relative, and gives no separate numbers for the real robot.

CAVEAT ALFWorld is a text environment where symbolic state relations are easy to read off, so the monitor's accuracy probably depends on clean, enumerable predicates that perception-heavy or continuous domains do not provide. The abstract also leaves unclear whether the monitor was trained on data from the same distribution, and whether the gain comes from the learned monitor or simply from having explicit subgoal state specifications at all.

WHY IT MATTERS This is peripheral to your work. It is single-agent tabletop and household planning with no communication, no negotiation and no other agents. The only transferable idea is loose: representing maneuver progress as explicit desired-versus-actual state relations, with an ongoing, deviated or completed label. That could inform how a maneuver coordination protocol signals abort or completion, but the paper does nothing in that direction.

arXiv:2610.39319v1 PDF incremental